Assignment 2 – Summary

GitHub username	ishan-monie
Repository name	advanced-ml-hw2
Link to the code	https://github.com/ishan-monie/advanced-ml-hw2

All code is in this repository (Python 3.12). The Deep Q-Network is in `dqn.py` and is used for both games.

How to run the code

1. Install the packages: `pip install -r requirements.txt`. 2. Run the commands below from the repository folder. Every check in `test_dqn.py` prints PASS or FAIL with the evidence.

What	Command
Train CartPole (about 3 minutes on a CPU)	`python dqn.py --game cartpole --seed 0 --out runs/cartpole_seed0 --snapshots 10,30,60,100,200,400`
Train MsPacman (about 3 hours on one GPU card)	`python dqn.py --game pacman --seed 0 --device cuda --out runs/pacman_seed0`
Play the 500 test games	`python evaluate_models.py --game cartpole --model runs/cartpole_seed0/model_best.pt --out results/cartpole_eval.npz` `python evaluate_models.py --game pacman --model runs/pacman_seed0/model_best.pt --out results/pacman_eval.npz --device cuda`
Make all graphs (figures/)	`python make_plots.py`
Run the checks (about 5 minutes)	`python test_dqn.py`
Tuning and extra experiments for CartPole	`python sweep_cartpole.py`, `python diagnose_cartpole.py`, `python experiment_discount.py`

The finished runs, logs, chosen networks and test-game scores are saved in the repository (`runs/`, `results/`), so the graphs and checks work without training. Seeds are fixed; CartPole runs use 1 computer thread (the thread count can change a run).

What is in the repository

File or folder	What it holds
`dqn.py`	The Deep Q-Network: replay buffer, neural networks, target network, training loop, test games.
`make_plots.py`	All graphs, including the two non-standard ones (`figures/`).
`test_dqn.py`, `test_results.py`	The edge-case and behaviour checks (what each one does is in this report and printed when it runs).
`sweep_cartpole.py`, `diagnose_cartpole.py`, `experiment_discount.py`	CartPole tuning, why weak networks fail, the discount experiment (results in `.csv` files).
`runs/`, `results/`	Training logs and chosen networks; test-game scores.

Deep Q-Network

(a) CartPole-v1

Assumptions

- Network: the 4 state numbers go through two hidden layers of 128 units to 2 values, the estimated future reward of pushing left or right. Discount 0.95, Adam with learning rate 5e-4, Huber loss (squared for small errors, a straight line for big ones), and Double DQN (one network picks the best next push and the target network values it, which reduces over-estimation).
- Replay buffer: the last 50,000 steps; learning starts after 1,000 steps, then one update per step on a random batch of 64. Target network: a frozen copy of the network that is refreshed every 250 steps. Exploration: the chance of a random push falls in a straight line from 1 to 0.02 over the first 10,000 steps. Training: 800 games.
- Which network is kept: the network changes a lot from game to game, so the training reward is a poor guide. Every 10 games the network plays 30 greedy games from a fixed set of starting states and the best one is kept. These games are separate from the final 500 test games, which use a different random seed and no random pushes.
- A game stopped by the 500-step limit is not treated as a game end, because the pole would have kept earning reward. The main model is seed 0, chosen in advance; seeds 1 to 4 show the effect of luck.

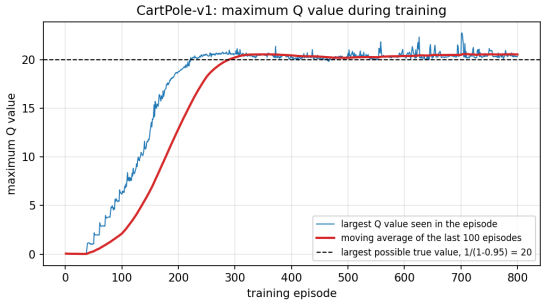


Figure 1. Largest Q value in each training game (blue) and its 100-game moving average (red). The true value of a balanced pole is at most 20.

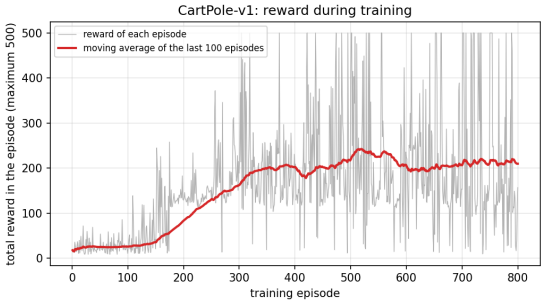


Figure 2. Reward of each training game and the 100-game moving average.

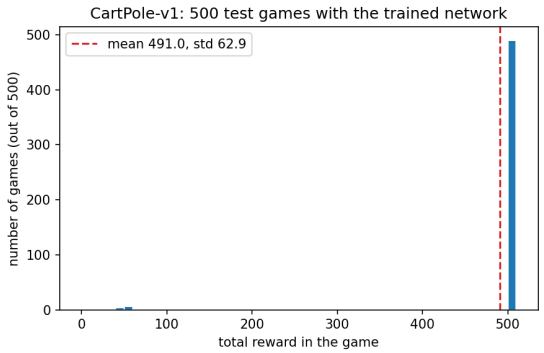


Figure 3. 500 test games with the chosen network (seed 0): mean 491.0, standard deviation 62.9.

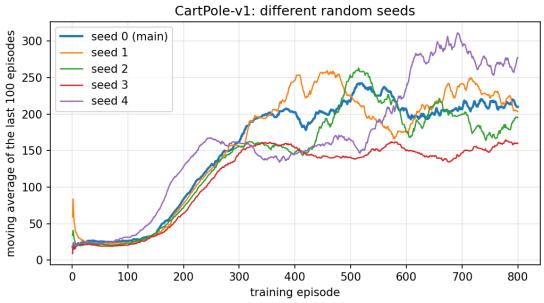


Figure 4. The five training seeds (100-game moving average of the training reward).

The network solves the game: in the 500 test games it scored a mean of 491.0 (standard deviation 62.9), and 98% of the games lasted the full 500 steps. The other seeds scored 498, 498, 378, 500. The largest Q value climbs to about 20, which is the true ceiling for a pole that is never lost (every step pays 1, so the value is 1/(1-0.95) = 20); it sits a little above that during training, which is the usual over-estimation of Q learning. The training curve (Figure 2) averages only about 210 because training games include random pushes and the network keeps changing; the chosen network plays much better than the training average suggests, and the network at the very end of training is much worse than the chosen one (see the tests), so the way the network is chosen matters.

Non-standard plot: what the network thinks the best push is

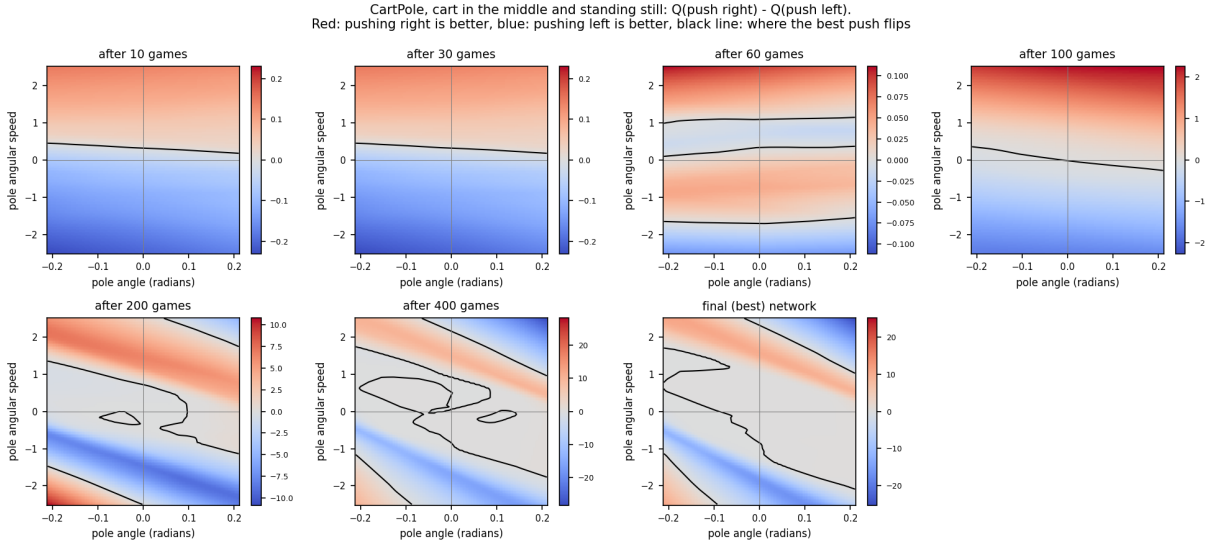


Figure 5. For every pole angle and pole speed (cart in the middle, standing still): the estimated value of pushing right minus pushing left, after 10, 30, 60, 100, 200 and 400 games and for the chosen network. After 10 and 30 games the network only reacts to the pole speed. By 200 to 400 games it has learned that a fast-falling pole needs a push the other way, and a band of states where the choice matters has formed.

Tuning

Our first CartPole runs scored only 100 to 250 and varied a lot. We tried 11 settings (learning rate, target update, buffer size, a share of game-ending steps in every batch, a falling learning rate, a bigger network) with 3 seeds each, then diagnosed the failures: in 2,013 of 2,262 lost test games of 25 end-of-training networks the cart had left the track and the pole had not fallen (a discount of 0.95 only looks about 20 steps ahead, so a slow drift towards the edge is barely visible). Choosing the network by greedy validation games instead of the training average was what mattered. The table shows the average score on 100 test games.

Setting	Network at the end, round 1 (3 seeds)	Chosen by training average, round 1	Network at the end, round 2 (5 seeds)	Chosen by validation games, round 2
hard target update every 500 steps	248	256	193	439
soft target update (the first try)	181	248	120	485
hard target update every 250 steps (used)	not run	not run	145	486

Setting	Network at the end, round 1 (3 seeds)	Chosen by training average, round 1	Network at the end, round 2 (5 seeds)	Chosen by validation games, round 2
plus 25% game-ending steps in every batch	294	184	201	429
plus a falling learning rate (1e-3 to 1e-5)	256	226	249	468

The settings are within noise of each other once the network is chosen by validation games (the same setting scores anywhere from 261 to 500 depending on the seed); the one with the highest average, a hard target update every 250 steps, was used for the final runs on 5 other seeds.

Edge cases

Edge case	How it was handled	How it was checked
A game stopped by the time limit versus a real failure: after 500 steps the pole would have kept earning reward.	Only a real failure is stored as a game end; a time-limit stop still counts the value of the next state.	With a time limit of 30 steps, games stopped by the limit were stored as not ended (8 games) and games where the pole fell as ended (52 games).
The network keeps oscillating: the network at the end of training is much worse than earlier ones.	The network is chosen by 30 greedy validation games every 10 games, from fixed starting states that are not the test games.	Seed 0: the chosen network scored 500 on 100 games, the last network 86.
Over-estimation: a Q learner tends to believe states are worth more than they are.	Double DQN, Huber loss and a target network that only changes every 250 steps.	The largest Q value settles at 20.5 against a true ceiling of 20 (highest 22.7); on the test games the average prediction was 19.95 against 19.99 actually earned.
The target network: if it moved with the network, or never moved, learning would break.	A frozen copy refreshed every 250 steps, never trained directly.	It stayed identical during 25 learning steps and equal to the network after a refresh; a version that never refreshed was caught on a tiny problem (error 1.05).
A discount of 0.95 only sees about 20 steps ahead, so drifting off the track is hard to learn.	Tried a falling learning rate, a share of failures in every batch, and a longer view (discount 0.99).	Cart left the track in 2,013 of 2,262 lost games (25 end-of-training networks); with discount 0.99 it fell from 473 to 279 games, and 102 games reached 500 (0 with 0.95).
Results that cannot be repeated.	Seeds are fixed, runs use 1 thread, and the test games use their own seed.	The same seed gave an identical training run; a different seed a different one; the saved network replays the saved test scores exactly.

Tests

Test	What was expected	What happened
The value each Q number is pulled towards, worked by hand: reward + discount x value of the next state, for 3 steps, the last one a game end, with and without Double DQN.	Code and hand calculation agree.	Identical (0.892, -0.259, and exactly the reward, 5, for the game-end step).
The first 10 steps of a real game, looked at one by one.	Start state near 0, reward 1 every step, each next state equal to the following state, only pushes 0 or 1.	All true (first state 0.014, -0.023, -0.046, -0.048).
A tiny problem with a known answer: a chain of 5 squares solved exactly, then learned by the same code.	The learned Q values match the exact ones.	Largest difference 0.000.
Can that test fail? A target network that is never updated, and a discount of 1 instead of 0.9.	Both should be caught.	Errors of 1.05 and 0.35 against 0.000.
Change the problem: discount 0.	Every CartPole state is worth just the next reward, 1.	All 4,000 Q values between 0.989 and 1.033.
Replay buffer: when full, the newest steps stay; batches are drawn evenly from the whole buffer.	Only the newest 10 of 25 steps remain; the age of drawn steps is spread evenly.	Newest 10 kept; average age 2,503 of 5,000 (even spread: 2,500), KS test p = 0.91.
Trained network against random play (500 and 100 games).	Far better (statistical test, t above 10).	491.0 against 23.9, t = 151.
Predicted against actual value in the test games.	Within 5% on average.	19.95 predicted, 19.99 actual.
Luck: 5 seeds.	All well above random, most near 500.	Test means 491, 498, 498, 378, 500.
Change the problem: discount 0.99 instead of 0.95 (5 seeds, network at the end of training).	Our prediction was that the average score would be 1.5 times higher. It was not (the policy still oscillates), but fewer games were lost by leaving the track.	Average 166 against 145; cart left the track 279 against 473; games reaching 500: 102 against 0.

All 25 checks for this game (16 general ones and 9 on the saved CartPole runs) are in test_dqn.py and test_results.py.

With 3 more months: CartPole

The scores depend on the checkpoint that is kept, so we would first find out why the policy oscillates (learning rate, target update interval, batch size) and try a stabler method such as prioritised replay, n-step returns or a dueling network. We would use 20 or more seeds with confidence ranges, test the trained network when the cart or pole is heavier or longer, measure over-estimation against exact values on small versions of the problem, and check how many steps of delay or noise in the state the network can handle.

(b) MsPacman-v5

Assumptions

- Input: the given `preprocess_observation` turns each screen into an 88 x 80 picture. The network sees the last 4 pictures stacked together (so it can tell which way things move); at the start of a game the first picture is repeated. The game (v5) repeats our previous action 25% of the time, skips 4 frames per step, has 9 actions, and ends when the last life is lost (losing one life is not treated as a game end).
- Network: three convolution layers (32 filters of 8 x 8 stepping 4, 64 of 4 x 4 stepping 2, 64 of 3 x 3) and one hidden layer of 512 units give 9 values, one per action (about 1.5 million weights). Discount 0.99, Adam with learning rate 1e-4, Huber loss, Double DQN, gradient size limited to 10.
- Replay buffer: the last 100,000 steps (stored as small whole numbers, about 5.6 GB); learning starts after 10,000 steps, then one update on a random batch of 32 every 4 steps. Target network: a frozen copy refreshed every 2,000 steps. Exploration: the chance of a random action falls in a straight line from 1 to 0.05 over the first 400,000 steps (about 800 games).
- Rewards: a pellet is worth 10 points and a ghost 200 or more, so for learning every reward is squashed to between -1 and +1. The scores in the graphs are the real game scores; the Q values in Figure 6 are in squashed units, where the highest possible value is 1/(1-0.99) = 100.
- Training: 6,000 games (about 4.6 million steps, 3.9 hours on one GPU card of the course server). The network that is kept is the one with the best 100-game moving average of the training reward; the test games play no part in that choice. The 500 test games use their own random seed and a 5% chance of a random action (a network that is always greedy can get stuck repeating the same moves), and were played on the CPU. The main model is seed 0, chosen in advance; seed 1 shows the effect of luck.

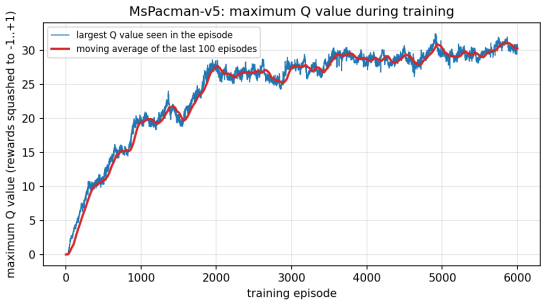


Figure 6. Largest Q value in each training game (blue) and its 100-game moving average (red), in squashed-reward units.

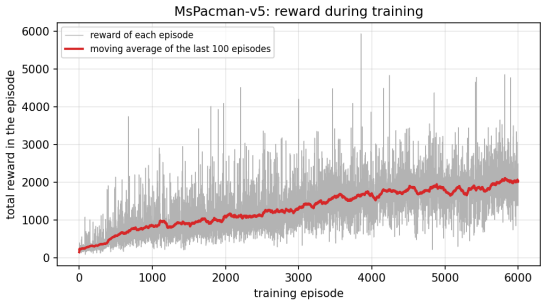


Figure 7. Score of each training game and the 100-game moving average.

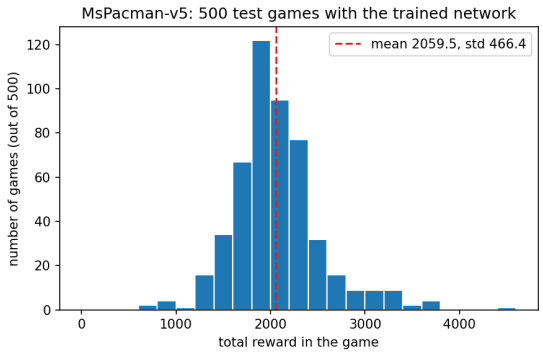


Figure 8. 500 test games with the chosen network (seed 0): mean 2059, standard deviation 466.

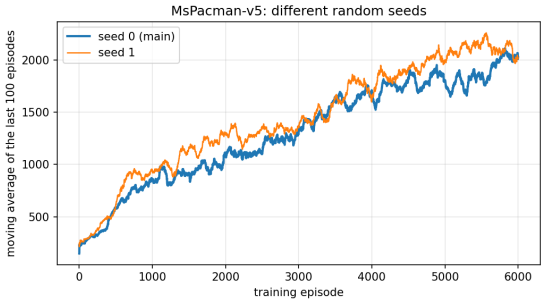


Figure 9. The two training seeds (100-game moving average of the training score).

The network improves steadily during training: the 100-game average goes from 274 in the first 100 games to 857 after 1,000, 1294 after 3,000 and 2024 at the end (seed 1: 286 to 2013). The curve is still rising slowly at 6,000 games, so more training would help. In the 500 test games the chosen network scored a mean of 2059 (standard deviation 466, lowest 680, highest 4530); the second seed scored a mean of 2088 (568). The largest Q value rises from about 0 to about 31, far below the highest possible value of 100, which fits a game in which a pellet gives only 1 squashed point.

Non-standard plot: predicted against actual value

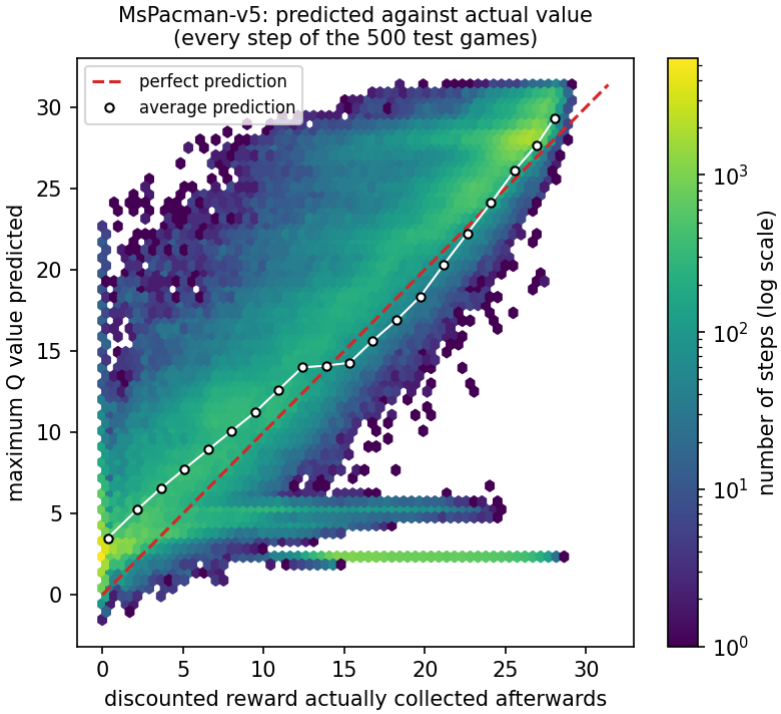


Figure 10. For every step of the 500 test games: the largest Q value the network predicted (vertical) against the discounted reward actually collected afterwards (horizontal), in squashed-reward units. On the dashed line the prediction would be exactly right. White dots are averages. The network over-estimates when little reward follows and under-estimates when a lot follows, which is typical: it cannot know in advance which games will go well.

Edge cases

Edge case	How it was handled	How it was checked
Rewards of very different sizes (10 points for a pellet, 200 or more for a ghost).	Rewards are squashed to between -1 and +1 for learning; reported scores are the real ones.	Largest stored reward 1.0, and real game scores are multiples of 10 (checked on 2 games).
One picture cannot show which way things move; pictures from two games must not mix.	Four pictures are stacked; a new game starts with the first picture repeated 4 times.	A new game started with 4 identical pictures; each step shifted the stack by exactly one picture; the game's own screen was left unchanged.
Randomness in the game (it repeats the previous action 25% of the time) and a greedy network getting stuck.	Test games use a 5% chance of a random action and their own seed; they were played on the CPU so that they can be replayed exactly.	The first 3 saved test scores were reproduced exactly from the saved network.
Over-estimation and unlimited growth of Q values.	Double DQN, Huber loss, a target network refreshed every 2,000 steps, and squashed rewards.	The largest Q value stayed near 31 (highest ever 32), far below 100.
Exploration: too little early on, or too much late.	The random-action chance falls in a straight line from 1 to 0.05 over 400,000 steps and then stays at 0.05.	First game 0.999; the minimum was reached at step 400,000 and kept to the last game.
A very large replay buffer, and a very long run.	Pictures are stored as small whole numbers; the log and the network are saved every 25 games; a time limit stops a run in time.	The buffer holds 100,000 steps (5.6 GB); both runs completed 6,000 games in 3.9 and 3.9 hours.

Tests

Test	What was expected	What happened
Pictures: the cleaned frame is 88 x 80 with values from -128 to 127; a new game starts with the first frame repeated 4 times; each step slides the stack by one frame; there are 9 actions.	All true.	All true.
Rewards: stored values between -1 and +1, reported scores real.	Largest stored reward 1; scores multiples of 10.	Largest stored reward 1; scores 150 and 160.
Trained network against random play (500 and 60 games).	Far better (statistical test, t above 10).	2059 against 258, t = 69
Can the results fail? A network with random weights.	It should score like random play, far below the trained network.	untrained network 384 (20 games), trained network 2059
Learning: last 100 training games against the first 100.	At least 3 times higher, and not by luck.	first 100 games 274, last 100 games 2024, p = 6.0e-61
Luck: the second training seed.	It also learns (at least 3 times higher).	seed 0: 274 -> 2024; seed 1: 286 -> 2013
Exploration schedule.	Starts near 1, reaches 0.05 only after 400,000 steps and stays.	As expected.
Predicted against actual value in the test games.	The predictions follow the actual returns (correlation above 0.5).	correlation 0.79; average prediction 14.4, average actual 13.3; highest prediction 31.4
The saved network replays the saved test scores exactly (first 3 games).	Identical.	replayed [1620.0, 2520.0, 1700.0], saved [1620.0, 2520.0, 1700.0]

The 9 checks on the saved MsPacman runs (and the 16 general ones, shared with CartPole) are in test_dqn.py and test_results.py; all 34 checks of the file pass.

With 3 more months: MsPacman

The score was still rising when training stopped, so the first step would be much longer training with several seeds and confidence ranges. We would add the improvements known to help Atari agents (prioritised replay, n-step returns, a dueling network) one at a time to see what each is worth, treat losing a life as a game end, and compare against the scores of random play and of a human player. We would also check how much the 4-picture stack and the reward squashing matter, test the trained network on different random starts and with the 25% repeated actions switched off, and look at what the network pays attention to in a picture.